

End-to-End ML Prediction Platform

Step-by-Step Development Roadmap for Cursor (AI Coding Assistant)

This document breaks the project into sequential phases. Each phase includes the goal, detailed steps, deliverables, recommended tech, and a ready-to-use prompt you can paste into Cursor to have it execute that phase.

Project summary: A reusable machine learning pipeline that accepts a dataset, trains and compares multiple ML models (regression / classification / probability tasks), selects the best-performing model, and serves predictions through a FastAPI backend with a simple web dashboard showing results, confidence, and model performance.

PHASE 1 Project Scoping & Planning

Goal: Define exactly what the platform will support before writing any code, so scope stays controlled.

What to do:

- Decide the initial set of supported prediction types: 1 regression task, 1 classification task, 1 probability/risk-scoring task (e.g. house price regression, customer churn classification, loan-default probability).
- Pick 2–3 public datasets (Kaggle/UCI) that match those tasks — e.g. House Prices, Telco Churn, Loan Prediction.
- Write a one-page spec: input features per task, target variable, models to compare, success metrics (RMSE/MAE for regression, F1/ROC-AUC for classification).
- Sketch the user flow: select task → upload/enter data → get prediction + confidence → view model performance page.
- List the final tech stack: Python, pandas, scikit-learn, XGBoost, FastAPI, React or simple HTML/JS frontend, SQLite/Postgres for logging, Docker for deployment.

Deliverables: spec.md, chosen datasets downloaded locally, wireframe sketch of the 3 main screens.

Cursor prompt for this phase:

Act as a senior ML engineer. Read my project spec (attached) and create a project folder structure for an end-to-end ML prediction platform with separate directories for data/, notebooks/, src/models/, src/api/, src/frontend/, tests/, and docker/. Generate a README.md skeleton and a requirements.txt with pandas, scikit-learn, xgboost, fastapi, uvicorn, pydantic, joblib, and pytest.

PHASE 2 Environment & Repository Setup

Goal: Get a clean, reproducible dev environment and a Git repo ready before touching data.

What to do:

- Initialize a Git repo and connect it to GitHub (repo name/description at the end of this doc).
- Create a Python virtual environment (venv or conda) and pin dependencies in requirements.txt.
- Set up a .gitignore for Python, data files, model artifacts, and .env secrets.
- Add a basic CI check (GitHub Actions) that runs linting and unit tests on push.
- Configure Cursor's project rules (.cursorsrules) describing the stack and coding conventions so generated code stays consistent.

Deliverables: GitHub repo initialized with README, .gitignore, requirements.txt, CI workflow file, .cursorsrules.

Cursor prompt for this phase:

```
Set up a Python project skeleton for a FastAPI + scikit-learn ML platform. Create a virtual environment config, requirements.txt, .gitignore for Python/ML projects, and a GitHub Actions workflow (.github/workflows/ci.yml) that installs dependencies and runs pytest on every push.
```

PHASE 3 Data Ingestion & Preprocessing Pipeline

Goal: Build a reusable, general-purpose preprocessing pipeline that works across the 3 chosen datasets/tasks.

What to do:

- Write a DataLoader module that reads CSVs and validates required columns per task.
- Handle missing values, outliers, and data type conversions.
- Build encoders for categorical variables (OneHot/Label/Target encoding) and scalers for numeric features (StandardScaler/MinMax).
- Create a train/validation/test split function with stratification for classification tasks.
- Persist the fitted preprocessing pipeline (scikit-learn Pipeline + joblib) so it can be reused at inference time.
- Write unit tests confirming the pipeline handles missing/malformed input gracefully.

Deliverables: src/preprocessing.py, saved pipeline artifacts (.joblib), unit tests, a short data-quality report per dataset.

Cursor prompt for this phase:

```
Create a reusable scikit-learn preprocessing pipeline in src/preprocessing.py that: (1) imputes missing values, (2) encodes categorical columns, (3) scales numeric columns, (4) is saved/loaded with joblib. Make it generic enough to accept a config dict specifying numeric_cols, categorical_cols, and target_col. Add pytest unit tests covering missing values and unseen categories.
```

PHASE 4 Exploratory Data Analysis (EDA)

Goal: Understand each dataset well enough to justify modeling choices, and produce visuals for the portfolio write-up.

What to do:

- Create a Jupyter notebook per task with distribution plots, correlation heatmaps, and target balance checks.

- Identify feature importance candidates and potential data leakage columns.
- Document key findings (e.g. skewed features needing log-transform, class imbalance needing SMOTE/class-weights).
- Export 3–4 key charts as images to use later in the README/demo.

Deliverables: notebooks/eda_.ipynb for each task, exported chart images, a findings.md summary.

Cursor prompt for this phase:

Generate an EDA Jupyter notebook template for a tabular ML dataset that includes: missing-value heatmap, numeric feature distributions, categorical value counts, correlation matrix, and target variable balance. Use pandas, matplotlib, and seaborn. Parameterize the dataset path and target column name at the top.

PHASE 5 Model Training & Comparison

Goal: Train and fairly compare multiple ML models per task so the platform can pick (or let the user pick) the best one.

What to do:

- Implement a ModelTrainer class supporting Logistic/Linear Regression, Random Forest, XGBoost (and optionally SVM/KNN).
- Use cross-validation and consistent metrics per task (RMSE/MAE/R² for regression, Accuracy/Precision/Recall/F1/ROC-AUC for classification).
- Run hyperparameter tuning (GridSearchCV or Optuna) for the top 2 candidate models per task.
- Log every experiment (params, metrics, training time) to a results table for comparison — MLflow is optional but recommended.
- Select and save the best model per task with joblib, including the metric that justified the choice.

Deliverables: src/models/train.py, saved model artifacts per task (.joblib/.pkl), results_comparison.csv, optional MLflow tracking folder.

Cursor prompt for this phase:

Write a ModelTrainer class in src/models/train.py that trains Logistic Regression, Random Forest, and XGBoost on a given train/test split, evaluates each with cross-validation, prints a comparison table of metrics, performs GridSearchCV on the top 2 models, and saves the best model with joblib along with a metadata JSON (model name, params, metrics, training timestamp).

PHASE 6 Model Evaluation & Interpretability

Goal: Make sure the chosen models are trustworthy and explainable, not just accurate on paper.

What to do:

- Generate confusion matrices/ROC curves for classification tasks, and residual plots for regression tasks.
- Add feature importance (built-in for tree models) and optionally SHAP values for explainability.
- Check for overfitting by comparing train vs test metrics.

- Write an `evaluation_report.md` per task summarizing final metrics and caveats.

Deliverables: evaluation plots (PNG), `evaluation_report.md` per task, optional SHAP summary plots.

Cursor prompt for this phase:

```
Write an evaluation script src/models/evaluate.py that loads a saved model and test set, then outputs: confusion matrix + ROC curve (classification) or residual + predicted-vs-actual plot (regression), plus a feature importance bar chart. Save all plots to reports/figures/ and print a metrics summary.
```

PHASE 7 Backend API Development (FastAPI)

Goal: Expose the trained models through a clean, documented REST API that the frontend and other clients can consume.

What to do:

- Design endpoints: `POST /predict/{task}`, `GET /models/{task}/metrics`, `GET /health`, `POST /upload-dataset` (optional retrain trigger).
- Define Pydantic request/response schemas per task matching the required input features.
- Load the saved preprocessing pipeline + model at startup and cache them in memory.
- Return prediction + confidence/probability + which model was used.
- Add input validation, error handling, and structured logging of every prediction request (for the monitoring feature later).
- Write API tests with `pytest` + `httpx TestClient` covering valid and invalid inputs.

Deliverables: `src/api/main.py`, `schemas.py`, routers per task, auto-generated OpenAPI docs at `/docs`, passing API tests.

Cursor prompt for this phase:

```
Build a FastAPI application in src/api/main.py with a POST /predict/{task} endpoint that: validates input against a Pydantic schema for that task, loads the corresponding saved preprocessing pipeline and model, returns { prediction, confidence, model_used, timestamp }, and logs the request to a local SQLite table. Add a GET /models/{task}/metrics endpoint returning the saved evaluation metrics JSON. Include error handling for missing/invalid fields and pytest tests using TestClient.
```

PHASE 8 Frontend / Web Dashboard

Goal: Give users a simple interface to submit inputs, see predictions, and browse model performance.

What to do:

- Build a task-selection page and a dynamic form generated from the task's required features.
- Call the FastAPI `/predict` endpoint and display prediction + confidence in a readable card.
- Add a model-performance page showing metrics, comparison chart, and feature importance for the active model.

- Style with a clean, minimal design (Tailwind or plain CSS) so it looks portfolio-ready.
- Handle loading and error states gracefully (bad input, server down, etc.).

Deliverables: frontend app (React or vanilla HTML/JS) with prediction page + metrics/dashboard page, connected to the live API.

Cursor prompt for this phase:

Create a simple React (Vite) frontend with two pages: (1) a Predict page that lets the user pick a task, renders a dynamic form based on that task's feature list, submits to the FastAPI `/predict/{task}` endpoint, and shows the prediction with a confidence badge; (2) a Model Performance page that fetches `/models/{task}/metrics` and displays them in a clean card layout with a bar chart (recharts). Use Tailwind for styling, keep it minimal and professional.

PHASE 9 Testing, Logging & Error Handling

Goal: Make the platform robust enough to demo live without breaking on edge cases.

What to do:

- Write end-to-end tests covering: valid prediction, missing field, wrong data type, unsupported task name.
- Add centralized logging (requests, errors, latency) to a file or lightweight DB table.
- Add basic rate limiting / input size limits to the upload endpoint.
- Test the full flow manually: frontend form → API → model → response → UI update.

Deliverables: tests/ folder with API + pipeline tests, logging configured, a short QA checklist completed.

Cursor prompt for this phase:

Add structured logging to the FastAPI app using Python's logging module: log every prediction request with timestamp, task, input summary, prediction, latency, and any errors to `logs/app.log` in JSON lines format. Then write pytest integration tests in `tests/test_api.py` covering valid predictions, missing fields, wrong types, and an unknown task name, asserting correct status codes and error messages.

PHASE 10 Deployment

Goal: Get a live, shareable version of the platform for the portfolio/resume link.

What to do:

- Containerize the backend with a Dockerfile (and the frontend if separate) and a `docker-compose.yml` for local runs.
- Deploy the API to a free/low-cost host (Render, Railway, or Fly.io) and the frontend to Vercel/Netlify.
- Set environment variables for production (CORS origins, API base URL).
- Verify the deployed app end-to-end and fix any CORS/env issues.
- Add a live-demo link and short GIF/screenshot to the README.

Deliverables: Dockerfile(s), docker-compose.yml, live API URL, live frontend URL, updated README with demo link.

Cursor prompt for this phase:

```
Write a multi-stage Dockerfile for the FastAPI backend (Python 3.11-slim base, install requirements, copy src/, expose port 8000, run with uvicorn) and a docker-compose.yml that runs the API and mounts the saved model artifacts as a volume. Also generate deployment configs/instructions for Render (backend) and Vercel (frontend), including the required environment variables.
```

PHASE 11 Documentation & Portfolio Polish

Goal: Package the project so a recruiter or client immediately understands what it does and why it's impressive.

What to do:

- Write a strong README: problem statement, architecture diagram, tech stack, how to run locally, live demo link, screenshots/GIF.
- Document the API with example requests/responses (or link to the auto-generated /docs).
- Add a short 'What I learned / challenges solved' section — this matters a lot to reviewers.
- Clean up commit history / squash messy commits if needed, tag a v1.0 release.
- Record a 60–90 second demo video or GIF walking through the app.

Deliverables: polished README.md, architecture diagram image, demo GIF/video, tagged v1.0 GitHub release.

Cursor prompt for this phase:

```
Generate a professional README.md for a GitHub ML project. Sections: Overview, Problem Statement, Architecture (with a placeholder for a diagram image), Tech Stack, Features, Live Demo link, Screenshots section, Local Setup instructions (backend + frontend), API Usage examples, Model Performance summary table, What I Learned, and License. Keep tone professional and concise.
```